

5. Testing and Quality Assurance

5.1 Strategy and Coverage Results

To ensure the reliability, security, and performance of the Dublin Bikes backend architecture, we implemented a robust automated testing strategy. Built using Pytest (v9.0.3) and pytest-cov, our approach encompassed both isolated Unit Tests and end-to-end Integration Tests for the Flask API layer.

To ensure hermetic testing environments, we configured `conftest.py` to route all test operations to an isolated in-memory SQLite database. This guaranteed fast execution and prevented any pollution of the production MySQL instance. The suite executed 355 test cases in just 12.5 seconds, achieving an exceptional 97% Statement Coverage across 1,272 lines of code.

Coverage Breakdown:

- API Routing & Data Contracts: 95% - 100%
- Journey, ML Prediction & Weather Services: 100%
- User Authentication Service: 96%
- AI Chat Service: 92%

5.2 Key Testing Scenarios and Methodologies

Achieving deterministic results required addressing external dependencies and complex boundary conditions:

- External API Mocking: We extensively utilized `unittest.mock.MagicMock` to intercept third-party network calls at the module level. We mocked the Google Maps client to test our "walk-cycle-walk" routing offline, stubbed OpenWeather to verify temporal filtering logic, and intercepted LLM interactions. This guaranteed 100% test determinism and prevented API quota exhaustion.
- ML Boundary Clamping: Using robust test fixtures (`make_weather_forecast`, `make_station`), we wrote explicit unit tests to guarantee that the Random Forest model's probabilistic outputs are always mathematically clamped between zero and a station's maximum dock capacity.
- Security Validation: To secure our JWT-based infrastructure, the suite simulated various unauthorized access attempts using custom `auth_headers` fixtures. We strictly tested token expirations, secret key mismatches, and time-to-live (TTL) boundaries to ensure correct HTTP 401 responses.